

Ejercicios programación orientada a objetos – relaciones simples

1) Escribe una clase **Punto** que tenga 2 atributos double llamados “x” e “y”, para que además de los constructores y de los métodos getX() y getY(), y distancia(), contenga también los siguientes métodos:

- Dos métodos para modificar los valores atributos.
- Un método toString() que devuelva la información del Punto de la siguiente manera (x, y).

Ayuda: el método toString() concatena los paréntesis y comas como literales con los valores de los atributos, en el orden requerido.

Ahora, utilizando la clase **Punto** anterior, escribe una clase **Polígono**. Esta clase tiene como atributo un array de objetos Punto. Se consideran adyacentes los objetos Punto que estén en celdas consecutivas del array y los puntos que están en la primera y última celda. Esta clase ha de tener los siguientes métodos:

- el constructor, que recibirá por parámetro un array con los objetos Punto que definen el Polígono.
- trasladar(), recibe por parámetro el desplazamiento en la coordenada x y el desplazamiento en la coordenada y.
- escalar(), recibe por parámetro el factor de escala para la coordenada x y el factor de escala para la coordenada y.
- numVértices(), devuelve el número de vértices del Polígono.
- perímetro(), devuelve el perímetro del Polígono.

Finalmente, implementa un programa principal donde pruebes el funcionamiento de todos los métodos desarrollados. Programa la semántica requerida para que la relación entre Polígono y Punto sea una agregación. Luego, duplica el proyecto y haz las modificaciones necesarias para que se comporte como una composición.

Algunos consejos:

- En el constructor hay que controlar que el número de vértices del Polígono no es menor que 3. En caso contrario, se lanza una excepción.
- Para trasladar el Polígono, se recorre el array desplazando cada uno de los puntos.
- Para escalar, se hace lo mismo, pero multiplicando su coordenada x por el factor x y su coordenada y por el factor y.
- El perímetro se calcula utilizando el método distancia() de la clase Punto. Se calcula y se suma la distancia de cada Punto con su Punto siguiente, sin olvidarse de la distancia entre el Punto de la celda 0 y de la última celda.

2) Desarrolla una aplicación que usará 2 clases: **Producto** y **Almacen**. La clase Producto tendrá como atributos: idProducto (int), nombre (String), precio (float) y cantidad (int). La clase Almacen tendrá como atributos: idAlmacen (int), nombre (String), un ArrayList de tipo Producto y capacidad (int), la cual no debe sobrepasarse sumando las cantidades de todos los productos del ArrayList. En este caso, queremos que la relación implementada permita mover productos de un almacén a otro, por lo que son objetos “independientes”.

En el programa principal, implementa un menú que permita ejecutar las siguientes acciones (deben estar implementadas también): alta de almacén, alta de producto, aumentar precio de todos los productos de un almacén, cambiar precio de un producto, mostrar datos producto, mostrar contenido

almacén, mover producto entre almacenes, buscar producto y salir. En el caso de buscar producto, lo tendrá que hacer por su “nombre” y tendrá que indicar en qué almacén se encuentra, o si no existe. En “mover producto entre almacenes” tendrá que pedir ID de producto, y, si existe, pedir a qué otro almacén debe moverse (sin mostrar en el que se encuentra), si la capacidad lo permite se moverá, y si no, no.

Para probarlo, debes, al menos, crear 2 almacenes inventados y 3 productos inventados. Asigna los 2 productos a un almacén, luego muévelos al otro. Muestra el contenido de cada Almacén antes del movimiento y después.